Architecture & flux de données

Architecture · PDF Equilibrist

<- Index · Contributing · Operations · Build

Vue d'ensemble

PDF Equilibrist est un éditeur PDF desktop construit sur PyQt6 (interface graphique) et PyMuPDF / fitz (moteur PDF). L'architecture suit une séparation stricte entre logique métier et interface utilisateur.

```
src/pdf_equilibrist/
··· main.py          # Point d'entrée
··· app.py           # Configuration QApplication + thème
··· utils.py         # resource_path() · chemins dev/exe
··· registration.py  # Enregistrement "Ouvrir avec..." Windows
··· core/
·   ··· document.py  # Modèle Document · seul détenteur du fitz.Document
··· ui/              # Couche présentation (PyQt6)
·   ··· main_window.py
·   ··· title_bar.py
·   ··· viewer.py
·   ··· thumbnail_panel.py
·   ··· floating_item.py
·   ··· page_edit_widget.py
·   ··· splash_screen.py
·   ··· signature_dialog.py
·   ··· batch_dialog.py
·   ··· widgets.py
·   ··· dialogs.py
·   ··· tabs/
·       ··· tab_afficher.py
·       ··· tab_modifier.py
·       ··· tab_convertir.py
·       ··· tab_annoter.py
·       ··· tab_page.py
·       ··· tab_proteger.py
··· operations/      # Logique PDF pure · sans UI, testable
    ··· edit.py
    ··· pages.py
    ··· annotate.py
    ··· convert.py
    ··· protect.py
```

Principe central : le signal `changed`

Document hérite de QObject et émet changed après toute modification.
Tous les composants UI s'y abonnent pour se rafraîchir automatiquement.

```
Toute opération PDF
    ··· document.changed.emit()
            ··· PdfViewer._on_doc_changed()    · re-render des pages
            ··· ThumbnailPanel.rebuild()       · re-render des miniatures
            ··· MainWindow._on_doc_changed()   · barre de statut, titre onglet
```

Règle : les fonctions de operations/ modifient fitz.Document en place.

Le tab appelant est responsable d'émettre document.changed après l'appel.

Flux de données typique

```
Utilisateur clique "Filigrane" dans tab_modifier
    .
    .
tab_modifier._add_watermark()
    ·  ask_watermark_text() · "CONFIDENTIEL"
    .
operations.edit.add_watermark(document.fitz_doc, "CONFIDENTIEL")
    ·  modifie fitz.Document en place (insert_text sur chaque page)
    .
document.changed.emit()
    ··· viewer.refresh()       · re-render QLabel de chaque page
    ··· thumbs.rebuild()       · re-render miniatures
```

Gestion multi-documents

MainWindow maintient une liste de Document, un par onglet ouvert.

```
_documents : list[Document]   # un Document par onglet
_active_idx : int             # index du doc affiché
_current_doc : Document       # référence directe (rebindée par _rebind_doc)
```

`_rebind_doc(doc)` · méthode centrale

Appelée à chaque changement d'onglet. Elle :

1. Déconnecte tous les signaux de l'ancien document (old.changed.disconnect())
2. Reconnecte le nouveau sur les 3 slots : _on_doc_changed, viewer._on_doc_changed, thumbs.rebuild
3. Met à jour la référence document dans chaque tab du ribbon
4. Force un refresh immédiat du viewer et des miniatures

Le viewer et le panneau miniatures sont partagés entre tous les onglets,
rebindés à chaque switch. Ils ne sont pas recréés.

Fenêtre frameless

MainWindow hérite de QWidget (pas QMainWindow) avec FramelessWindowHint.

```
MainWindow (QWidget, FramelessWindowHint)
··· TitleBar              · drag via startSystemMove()
··· QMenuBar              · menu Fichier
··· Ribbon (QTabBar + QStackedWidget)
··· QSplitter
·   ··· ThumbnailPanel    · largeur fixe adaptive
·   ··· PdfViewer         · stretch=1, prend l'espace restant
··· Status bar
```

Redimensionnement : mousePressEvent sur les bords appelle windowHandle().startSystemResize(edge) · API Qt6 native.

Déplacement : TitleBar.mousePressEvent · startSystemMove() · natif.

PowerToys FancyZones / Windows Snap : showEvent appelle _apply_win32_styles() qui ajoute WS_THICKFRAME | WS_MAXIMIZEBOX | WS_MINIMIZEBOX via SetWindowLong. Cela rend la fenêtre reconnaissable par PowerToys et active Windows Snap (Win+flèche).

Snap Layout Windows 11 : nativeEvent intercepte WM_NCHITTEST et retourne HTMAXBUTTON quand le curseur survole le bouton maximiser · Windows affiche le popup de disposition Snap. WM_SYSCOMMAND est aussi intercepté pour SC_MAXIMIZE / SC_RESTORE / SC_MINIMIZE envoyés par Snap et PowerToys.

> Sécurité mémoire : ctypes.string_at + from_buffer_copy copie les bytes
> du MSG struct dans un buffer Python avant lecture, évitant le segfault de
> from_address qui lit en place une adresse potentiellement invalide.

Mode édition de texte

```
1. extract_text_blocks(doc, page_i)
       ·· get_text("dict") · liste de TextBlock
              (rect, origin baseline, fontname, fontsize, color, font_buffer)

2. viewer.enter_edit_mode(blocks_by_page)
       ·· remplace QLabel · PageEditWidget par page

3. Clic sur un bloc · _BlockEditor (QTextEdit inline)
       ·· positionné pixel-perfect sur span["origin"] (point baseline)

4. Entrée (validation) · apply_text_edits()
       ·· add_redact_annot(rect, fill=None)  · supprime sans rectangle blanc
       ·· apply_redactions()
       ·· insert_text(origin, new_text, ...)  · réinsère au baseline exact

5. viewer.exit_edit_mode() · refresh()
```

Point clé : fill=None dans la redaction évite le rectangle blanc visible sur les pages avec fond coloré ou image en arrière-plan.

Placement flottant (texte / image / tampon)

```
viewer.show_floating_item(data, page_index, on_commit, on_cancel)
    ·· FloatingItem(data, zoom, page_widget)
          ·· drag · déplacement (startSystemMove-like manuel)
          ·· coins · redimensionnement
          ·· poignée verte · rotation (atan2 autour du centre)
          ·· "· Valider"
                ·· _commit()
                     ·· pdf_rect = screen_coords / zoom
                     ·· committed.emit({...pdf_rect, angle})
                            ·· tab._on_placement_commit()
                                   ·· page.insert_text/insert_image/draw_rect
                                          ·· document.changed.emit()
```

Panneau miniatures · DnD

Le drag & drop est géré par ThumbnailPanel (le panneau, pas la liste Qt).

- Drag interne : event filter sur viewport · drag_started(row) · QDrag avec mime application/x-pdf-equilibrist-row · doc.move_page(src, to)
- Drop PDF externe : dragEnterEvent sur panneau accepte les URLs .pdf · doc.insert_pdf(src_doc, start_at=insert_before)
- Barre d'insertion : widget _DropLine transparent aux events, posé sur le viewport, dessine la ligne blanche à la position d'insertion calculée

Panneau miniatures · sélection multiple & menu contextuel

_ThumbList est en SelectionMode.ExtendedSelection : Ctrl-clic ajoute/retire une page, Shift-clic sélectionne une plage (comportement standard Qt).

ThumbnailPanel._on_right_click() calcule la sélection courante (clic droit sur une page hors sélection · ne garde que cette page ; clic droit sur la sélection courante · la préserve) et construit un menu unique dont toutes les actions opèrent sur la liste de pages sélectionnées, qu'il y en ait une ou plusieurs :

- Monter / Descendre (_move_selection) : traite les pages triées en ordre croissant (monter) ou décroissant (descendre) via doc.move_page() successifs · fonctionne aussi pour une sélection non contiguë.
- Rotation horaire / antihoraire (_rotate) : operations.pages.rotate_pages(doc, angle, rows).
- Dupliquer (_duplicate) : copie le bloc sélectionné juste après la dernière page sélectionnée.
- Extraire· (_extract) : operations.pages.extract_pages() + dialogue "Enregistrer sous" · écrit un nouveau PDF, le document ouvert n'est pas modifié.
- Supprimer (_delete) : désactivé si la sélection couvre toutes les pages.
- Imprimer· (_print_selection) : ouvre PrintDialog avec initial_pages=rows, qui pré-remplit "Pages personnalisées".

Chaque action mutante appelle document.checkpoint() avant modification (pile d'annulation Ctrl+Z).

Impression (Ctrl+P)

```
Ctrl+P / bouton Imprimer / Fichier · Imprimer
    ··· MainWindow._print()
         ··· ui/print_dialog.print_document(document, parent)
              ··· PrintDialog                       · aperçu, imprimante, pages, échelle
              ··· _qt_print()                       · Linux/macOS : QPrinter/QPainter (CUPS)
              ··· _win32_print()                    · Windows : Win32 GDI directe
                   ··· StartDoc / StartPage
                   ··· fitz.get_pixmap(dpi/UPSAMPLE) · rendu à demi-résolution
                   ··· StretchDIBits HALFTONE ×2     · upscale · traits fins visibles
                   ··· _draw_vector_strokes()        · overlay vectoriel GDI
                        ··· get_drawings(extended=True) · tous les paths PDF + XObjects
                        ··· Polyline GDI (chaînes)      · jonctions propres, sans pâtés
                        ··· filtres hachures            · clip / multi-segments / image
```

Stratégie hairline (plans AutoCAD)

Les traits fins (0·0.25 pt) rastérisés à pleine résolution imprimante deviennent invisibles. Solution en deux temps :

1. UPSAMPLE=2 : PyMuPDF rend à dpi/2, puis StretchDIBits HALFTONE upscale ×2. Chaque pixel fitz devient 2×2 px sur le papier · traits fins visibles.
2. Overlay GDI : _draw_vector_strokes() re-trace les paths stroke-only en GDI par-dessus le raster pour des jonctions nettes et des largeurs normalisées.

`_draw_vector_strokes()` · filtres appliqués

Filtre	Condition	Cible
Clippath	`clip is not None`	Hachures AutoCAD avec clippath
Multi-lignes	`len(items) > 30` tous `l`	Hachures dans un seul path
Pas de stroke	`color` absent	Fills solides, polylignes larges
Image large	image > 50 % page	Photo aérienne, fond scanné
< 500 drawings	total drawings < 500	Excel, Word, PDF simples

Les paths l (ligne), re (rect), qu (quad), c (bézier cubique) sont tous gérés. Les béziers sont approchés en 8 segments de ligne.

Compatibilité PyInstaller

resource_path(relative) résout les chemins assets :

```
# Dev  : src/pdf_equilibrist/utils.py · remonter 3 niveaux · racine projet
# Exe  : sys._MEIPASS · dossier d'extraction temporaire PyInstaller
```

Tous les accès aux assets (logo, boutons, splash) passent par resource_path().

Sécurité · Vérificateur CVE (`cve_checker.py`)

Accessible depuis le menu Aides · Vérifier les vulnérabilités CVE.

```
CVEDialog
    ··· _ScanThread (QThread)
          ··· cve_checker.get_installed_packages()
          ·        ··· importlib.metadata.distributions()
          ·                ··· {nom: version} pour chaque paquet installé
          ·                    (fonctionne en exe PyInstaller sans requirements.txt)
          ··· cve_checker.scan_dependencies(pkgs)
          ·        ··· ThreadPoolExecutor(max_workers=8)
          ·                ··· query_package_vulnerabilities(name, version)
          ·                         ··· POST https://api.osv.dev/v1/query
          ·                              {package: {name, ecosystem:"PyPI"}, version: "x.y.z"}
          ··· progress.emit(done, total) · barre de progression X/N paquets
```

Clé de conception : l'envoi de la version à OSV.dev limite les résultats aux CVE affectant précisément la version installée, éliminant les faux positifs des versions corrigées. Les 8 requêtes HTTP parallèles réduisent le temps de scan de ~N×t à ~t.

Mises à jour et statistiques (`update.py` + `ui/update_dialog.py`)

Accessible depuis Aides · Vérifier les mises à jour.

```
UpdateDialog
    ··· _CheckThread (QThread)          · démarre en parallèle avec _StatsThread
    ·        ··· updater.get_latest_release_info(__version__)
    ·                ··· GET https://api.github.com/repos/{owner}/{repo}/releases/latest
    ·                        ··· compare tag_name avec __version__ (tuple)
    ·
    ··· _StatsThread (QThread)
             ··· updater.get_download_stats(repo, __version__)
                      ··· GET https://api.github.com/repos/{owner}/{repo}/releases?per_page=100
                              ··· cumule asset.download_count :
                                   "current" : release dont tag == v{__version__}
                                   "total"   : toutes releases confondues
                                   "releases": nombre de releases
```

L'interface affiche deux compteurs live :

- Cette version (vert #6BBF4E) · téléchargements de la version courante
- Total (blanc) · cumul de toutes les releases

Les deux threads démarrent simultanément dans __init__ ; les compteurs s'affichent dès que _StatsThread retourne, indépendamment du résultat de _CheckThread.

Technologies

Composant	Librairie	Rôle
Interface	PyQt6 6.11	Fenêtre, widgets, signaux
Moteur PDF	PyMuPDF 1.27	Rendu, édition, annotations
PDF · Word	pdf2docx 0.5	Conversion structurelle
PDF · Excel	pdfplumber + openpyxl	Extraction tableaux
PDF · PPT	python-pptx	Slides image
Office · PDF	docx2pdf / LibreOffice	Automation COM ou headless
Images	Pillow 12	Manipulation images
Build	PyInstaller	Dossier onedir Windows (exe + `_internal
CVE scan	importlib.metadata + OSV.dev	Vulnérabilités dépendances
Mises à jour	GitHub Releases API	Vérification + stats téléchargements